

Level 0 — A little something to get you started

Vulnerability Report

Program: Hacker101 CTF

Target: Level 0 — A little something to get you started

Findings: 1 confirmed

Executive Summary

This report documents a single confirmed vulnerability discovered during assessment of the Level 0 challenge application. The application exposed a sensitive resource through a client-visible reference in page styling metadata. An unauthenticated visitor who inspected the rendered HTML could discover the resource path and retrieve privileged content by requesting that path directly.

The issue is primarily a failure of authorization and information-hygiene controls around how privileged assets are referenced and served. Remediation centers on removing privileged paths from public responses and enforcing access control on every request for sensitive resources.

Detailed offensive walkthrough notes for this level are retained in `CTFs/Raw_Notes/`. This report focuses on validated impact and remediation.

Finding 1: Sensitive Resource Disclosure via Client-Visible Asset Path

Field	Value
Severity	Medium
Vulnerability Class	Information Disclosure / Missing Access Control
CWE	CWE-200 (Exposure of Sensitive Information), CWE-425 (Direct Request / Forced Browsing)

Description

The application's public HTML response included a style-related reference to a backend asset path. The referenced asset did not present as a normal, visible image on the page, which made the reference easy to overlook during casual browsing. However, the path itself was fully available to any client that inspected the page source.

Because the privileged content was reachable by a direct HTTP request to that path, discovery of the reference was sufficient to retrieve the sensitive resource. In other words, confidentiality of the resource depended on obscurity of the URL rather than on an authorization check.

In a production CMS or static-hosting setup, this pattern commonly appears when internal or admin-only files are linked from CSS `background` URLs, hidden image tags, commented markup, or other client-delivered metadata without a gatekeeping layer.

Proof of Concept

Validation summary (non-actionable):

1. The public challenge page was loaded in a standard browser.
2. Client-delivered HTML was reviewed. A style element in the document header referenced a filename/path that did not correspond to a visibly rendered image.
3. Requesting that same path relative to the application origin returned privileged content rather than an authorization failure.
4. Successful retrieval of the sensitive resource was confirmed by the following flag value:

```
^FLAG^84d3c2462e4f813cc55719e8be80d6788c7b4cde201235138b3df35698691104$FLAG$
```

No authentication or elevated role was required.

Impact

- **Confidentiality breach:** Sensitive content intended to remain hidden was available to any unauthenticated visitor who inspected client-side references.
- **Low exploitation barrier:** The only prerequisites were viewing page source and issuing a normal GET request.
- **Real-world analogue:** Similar defects can expose admin panels, backup files, feature flags, internal APIs, or unreleased assets when paths leak into CSS, JavaScript bundles, HTML comments, or error pages.

Business impact in a live deployment would scale with the sensitivity of the disclosed resource (PII, credentials, proprietary content, etc.).

Mitigation

1. **Never embed privileged resource paths in public responses.** Do not reference admin-only, secret, or gated files from CSS, inline styles, HTML attributes, or JavaScript delivered to unauthenticated users.
2. **Authorize every request.** Serve sensitive files through an application handler that checks session/role before streaming content. A correct guess of a URL must not be sufficient.
3. **Use non-guessable, non-descriptive identifiers only as a secondary control,** and still enforce authorization. Opaque filenames are not a substitute for access control.
4. **Prefer authenticated storage patterns:** signed short-lived URLs, object-storage policies, or server-side templates that inject asset URLs only after authorization succeeds.
5. **Harden discovery surface:** remove unused style/asset references; ensure 404 vs 403 behavior does not unnecessarily confirm the existence of sensitive objects to unauthorized callers when that confirmation itself is sensitive.
6. **Regression testing:** add checks that public HTML/CSS/JS responses do not contain known privileged path prefixes, and that direct requests to those resources without auth return a uniform denial.

Affected Assets

- Public landing page HTML (header `<style>` / background asset reference)
- Privileged resource reachable by direct request to the disclosed path on the challenge origin

Appendix: Approaches Tried

The following probes were performed during assessment and did **not** independently yield the flag. They are included to document thoroughness and to clarify what was *not* the vulnerability surface.

Approach	Result
Initial HTML source review for obvious embedded secrets	No immediately usable flag string observed on first pass
Interpreting hex-encoded values present in links as the flag	Not the flag
Treating visible on-page text as the flag	Not the flag
Re-inspection of header style metadata and direct retrieval of the referenced path	Confirmed finding (see Finding 1)

These negative results reinforce that the issue was not “flag in visible copy,” but unauthorized reachability of a resource whose path leaked through client-visible styling metadata.